

Technical Requirements Document

Job Search Automation Platform - JobPilot

Version	1.0 technical draft
Date	2026-06-19
Source	Joblist v1 PRD draft plus updated owner process flow
Capacity target	1-50 candidates maximum in the initial operating model
Initial portal	Naukri first; LinkedIn, Workday, and Indeed later
Budget posture	Free services wherever practical; paid spend only for hosting or explicitly approved services

Technical Position

The MVP should be a small scheduled backend service, not a large multi-worker platform. Use Google Forms and Sheets for intake, Google Drive for resume files, SQLite for operational metadata, a Naukri scraping/ranking agent for the first release, and email delivery for reports. Scale by adding worker queues only after actual demand exceeds the 1-50 candidate target.

Document Purpose

This document translates the JobPilot PRD into implementable technical requirements, system architecture, data contracts, scheduler behavior, agent responsibilities, deployment choices, risk controls, and acceptance criteria. It is written for building an MVP with minimal budget while leaving clear extension points for additional job portals.

1. Scope and Operating Constraints

JobPilot is a backend-only job discovery automation service. Candidates submit a Google Form once, upload a PDF resume, and receive recurring Excel reports by email. The initial release supports a small private user group, not a public SaaS product.

MVP Scope

- Google Form intake with responses stored in a linked Google Sheet.
- Google Drive resume storage for uploaded PDF resumes.
- Scheduled backend row picker that identifies active candidates due for a run.
- Parallel profile-building tasks: form-data normalizer and PDF resume parser.
- Merged Candidate Profile Object used by the Naukri job discovery and ranking agent.
- Excel report generation with job details, source, match score, and apply links.
- Email delivery to the candidate using the email captured in the form.
- Operator logs, failure alerts, and manual run commands for support.

Explicit Non-Goals for MVP

- No auto-apply workflow.
- No candidate-facing dashboard or payment system.
- No multi-tenant admin UI in the first release.
- No 500-user concurrency target; the architecture is designed for 1-50 candidates.
- No paid resume parsing API or paid LLM dependency unless approved later.

Capacity Adjustment from PRD

The PRD mentioned 500 concurrent candidate pipelines. This TRD replaces that target with a maximum of 1-50 candidates. The engineering design therefore favors a single small server, low concurrency, simple retries, and clear logs over distributed infrastructure.

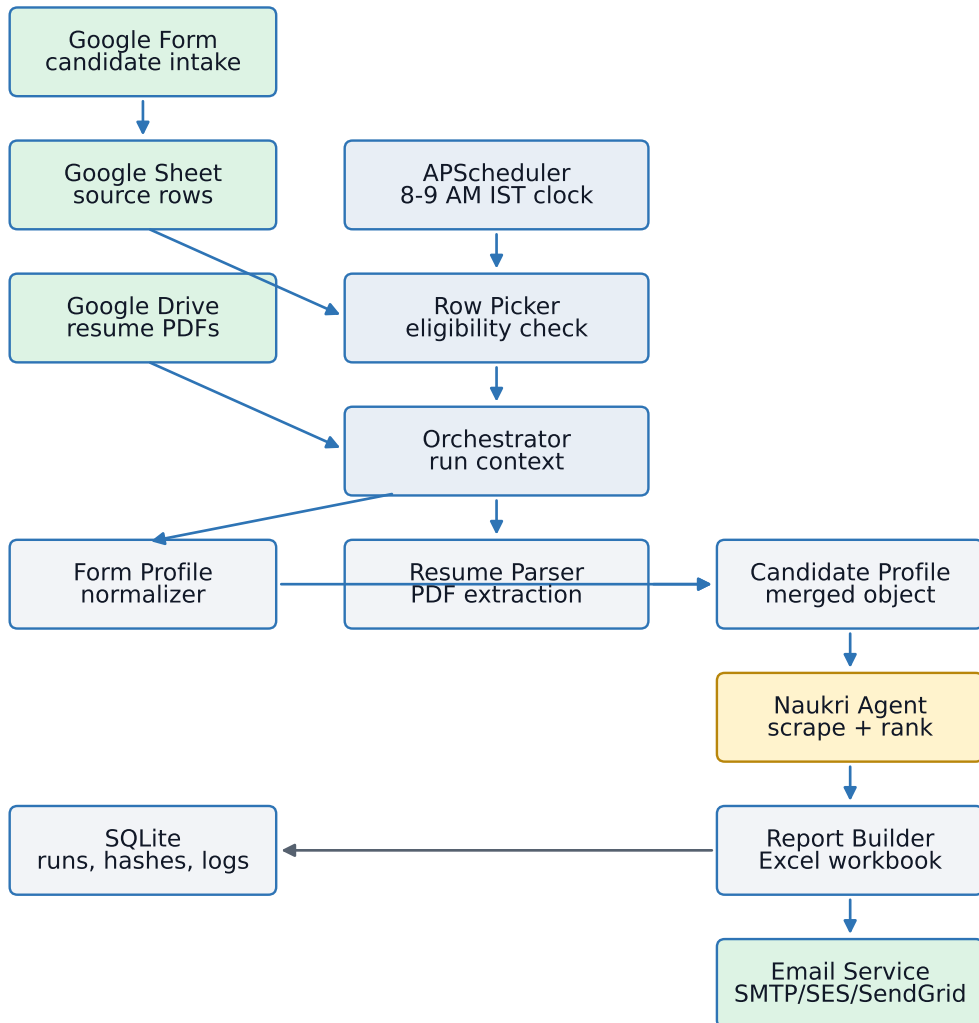
2. Recommended Low-Cost Stack

Layer	Recommended MVP Choice	Reason
Language	Python 3.11+	Strong ecosystem for scraping, Sheets integration, PDF parsing, and Excel output.

Scheduler	APScheduler inside backend process	Simple for one server and 1-50 candidates; avoids Celery until needed.
Intake	Google Forms + Google Sheets	Free/low-cost and non-technical for candidates.
File Store	Google Drive via Form upload	Keeps resumes outside the app database.
Operational DB	SQLite	Enough for run state, report history, dedupe hashes, and errors at this scale.
Scraping	Playwright + BeautifulSoup	Browser automation where needed, parser cleanup after page load.
Resume Parsing	pdfplumber or PyMuPDF	Free local parsing for text-first resumes.
Report	pandas + openpyxl/xlsxwriter	Creates structured Excel workbooks with links and multiple sheets.
Email	Gmail SMTP app password, SendGrid free tier, or AWS SES	Start free/simple; move to SES for higher reliability if needed.
Hosting	Render/Railway/Fly.io small instance or low-cost VPS	One always-on worker is sufficient for scheduled backend work.

Technical Architecture Pipeline

The MVP uses Google Workspace as the intake and file store, a scheduled Python backend as the orchestrator, parallel profile-building tasks, and one production portal agent for Naukri. Later portals are plugged into the same agent interface without changing the intake, reporting, or email layers.

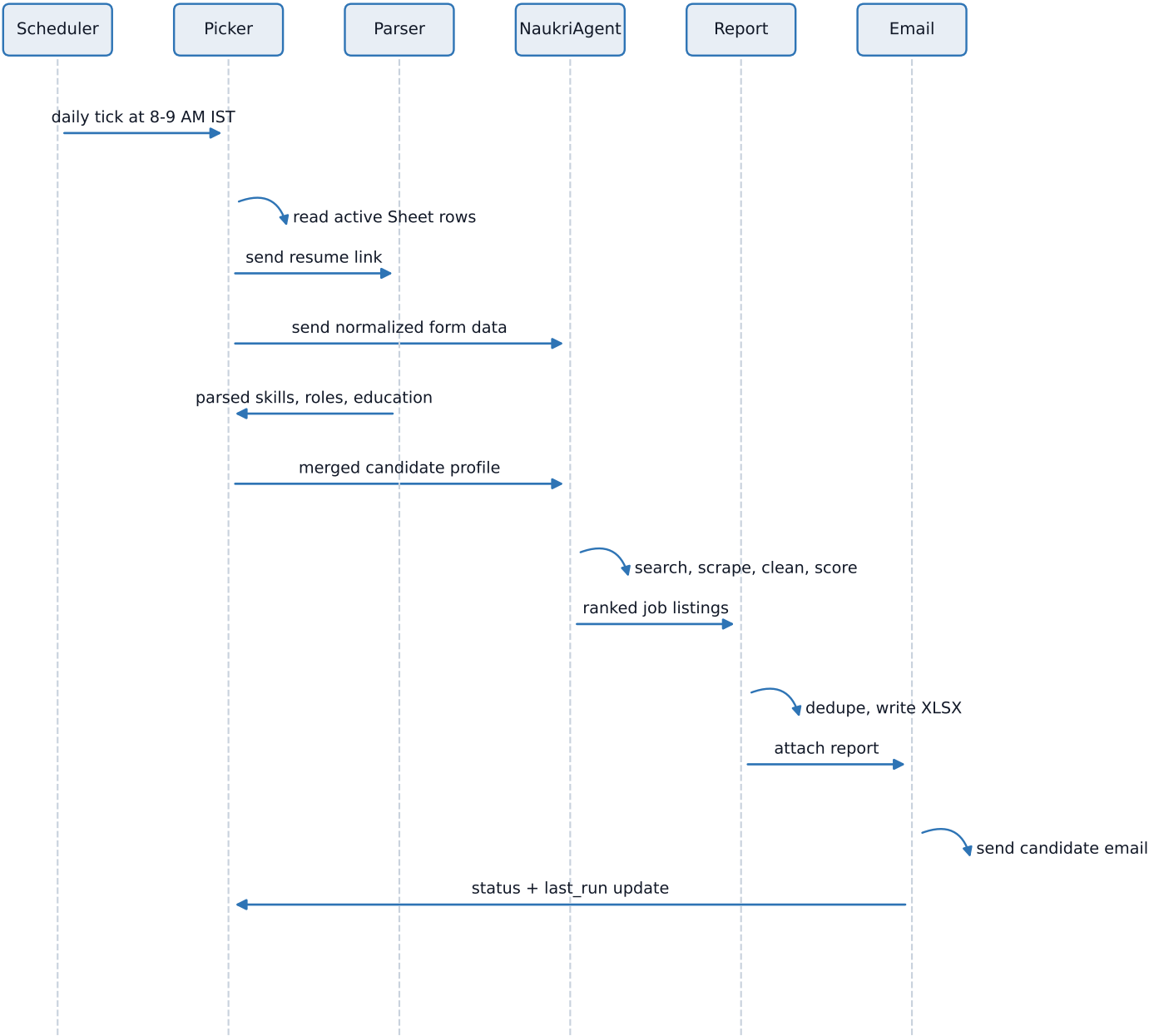


Future agent slots: LinkedInAgent, WorkdayAgent, IndeedAgent use the same input/output contract.

3. End-to-End Functional Flow

1. Candidate fills the Google Form and uploads a PDF resume.
2. Google Form writes a new response row into Google Sheets and stores the resume in Google Drive.
3. At the configured clock time, the scheduler starts the row picker.
4. The row picker reads active rows and checks whether each candidate is due based on frequency and next_run_at.
5. For each due candidate, the orchestrator creates a run_id and launches form profile normalization and resume parsing concurrently.
6. The parser extracts skills, roles, companies, education, projects, certifications, and inferred keywords from the resume.
7. The profile merger resolves conflicts between form fields and resume facts; form-submitted preferences win over inferred data.
8. The NaukriAgent builds search queries using roles, skills, experience, location, employment type, and salary preferences.
9. The agent scrapes listings, normalizes fields, removes duplicates, and scores each listing against candidate preferences.
10. The report builder creates an Excel workbook with an All Jobs sheet and a Naukri sheet.
11. The email service sends the workbook to the candidate and stores delivery status.
12. The backend updates the Google Sheet and SQLite metadata with last_run_at, next_run_at, counts, and errors.

Candidate Run Sequence



Failure rule: parser or portal failures create a partial report and an operator alert; they do not stop the whole run.

4. Scheduling Requirements

The scheduler must support both a fixed weekday cadence and an every-N-days cadence. The initial business preference is a morning run, around 8:00 AM to 9:00 AM India time. The exact mode should be configurable per deployment and optionally per candidate.

Requirement	MVP Behavior
Timezone	Asia/Kolkata for all scheduled decisions unless explicitly changed.
Default send window	8:30 AM IST, with allowed configuration between 8:00 AM and 9:00 AM.
Cadence option A	Monday, Wednesday, Friday.
Cadence option B	Every 3 days, e.g., Monday then Thursday then Sunday then Wednesday.
Candidate control	Sheet fields: active, frequency_mode, frequency_days, weekdays, last_run_at, next_run_at.
Manual trigger	Operator CLI or admin-only HTTP endpoint can trigger one candidate or all due candidates.
Missed run handling	If the server was down, run due candidates on next startup, then recompute next_run_at.
Concurrency cap	Run 1-3 candidates at a time; avoid launching 50 browser sessions together.

Recommended Default

Use Monday/Wednesday/Friday at 8:30 AM IST for predictable delivery. Keep every-3-days support in the scheduler model, but start with one default cadence unless a candidate explicitly needs another schedule.

5. Data Contracts

Google Form Fields

- Full name, email, phone, current city, preferred locations, current title, years of experience.
- Skills, preferred job roles or keywords, expected salary, notice period, employment type.
- Target portals, report frequency, consent checkbox, active/pause preference, and resume PDF upload.

- Optional future fields: excluded companies, preferred industries, work authorization, remote preference, and job seniority.

Google Sheet Operational Columns

Column	Purpose
candidate_id	Stable generated ID; do not depend only on row number.
active	Whether scheduled runs should process this candidate.
resume_file_id	Google Drive file ID extracted from the upload URL.
frequency_mode	weekday or every_n_days.
weekdays	Comma-separated list such as Mon,Wed,Fri.
frequency_days	Number such as 3 for every-three-days mode.
last_run_at	Last completed or attempted run timestamp.
next_run_at	Next due timestamp computed by scheduler.
last_report_url	Optional Drive link if reports are archived.
last_status	success, partial_success, failed, skipped.
last_error	Short operator-facing error summary.

6. Candidate Profile Object

All agents should consume one merged Candidate Profile Object. This prevents each portal agent from understanding Google Sheets, resume files, or raw form formats. It also makes LinkedIn, Workday, and Indeed easier to add later.

Field Group	Fields
Identity	candidate_id, full_name, email, phone, current_location
Preferences	target_roles, preferred_locations, employment_type, expected_salary, notice_period, target_portals
Experience	years_experience, current_title, past_roles, companies, seniority_guess
Skills	skills_from_form, skills_from_resume, normalized_skills, must_have_keywords
Resume	resume_file_id, resume_text_hash, parsed_sections, parse_confidence
Search Plan	query_variants, location_variants, filters, max_jobs_per_run

Conflict Rule

Candidate form preferences are authoritative for locations, salary, employment type, and target roles. Resume parsing may enrich skills and experience, but it should not override explicit candidate preferences without operator review.

7. Agent Requirements

Profile Builder and Resume Parser

- Download the PDF from Google Drive using a service account with limited access.
- Extract text and structured sections using local libraries first.
- Return parse_confidence and warnings for scanned/image-only resumes.
- Fallback to form data when PDF extraction is incomplete.
- Do not store raw resume text permanently unless explicit retention is approved.

NaukriAgent MVP

- Input: Candidate Profile Object.

- Build query variants from preferred roles, normalized skills, years of experience, and locations.
- Scrape listing search results with conservative rate limits and no account credential sharing.
- Normalize title, company, location, experience, salary, date posted, description snippet, source, and apply URL.
- Score each job using deterministic criteria first: role match, skill overlap, location match, experience fit, recency, and employment type.
- Return at most a configurable number of jobs per run, e.g., 50-100 jobs for Naukri.
- Log selector failures and portal layout changes clearly so maintenance is fast.

Future Portal Agents

Agent	Phase	Notes
LinkedInAgent	Phase 2	High compliance risk; prefer official or user-authorized routes if available.
WorkdayAgent	Phase 3	Likely needs company career-site discovery rather than one global Workday search.
IndeedAgent	Phase 3	Review terms and available publisher/API options before scraping.

8. Matching, Deduplication, and Report Output Scoring Model

Criterion	Weight	MVP Rule
Role/title match	30	Keyword overlap between target roles and job title.
Skill match	25	Overlap between normalized skills and job description/title.
Location match	20	Exact or nearby preferred location; remote is treated separately.
Experience fit	15	Candidate years fall within listed experience range or close tolerance.
Recency	10	Newer postings rank higher.

Use deterministic scoring in the first release. An LLM can later summarize descriptions or improve ranking, but it should be optional because the budget target is near zero.

Deduplication

- Within a portal, dedupe by normalized title + company + location + apply URL.
- Across future portals, dedupe by normalized title + company + location, with fuzzy matching for small spelling variations.
- Across runs, store job hash values in SQLite so already-seen jobs can be excluded or marked as previously sent.

Excel Workbook

Sheet	Columns
All Jobs	Rank, Match Score, Job Title, Company, Location, Experience, Salary, Type, Posted Date, Source, Apply Link, Notes
Naukri	Same as All Jobs, filtered to Naukri source.
Run Summary	Candidate name, run timestamp, search queries, total jobs found, skipped/failed agents.

9. Email Delivery Requirements

- Send email to the candidate address from the Google Form.

- Subject format: JobPilot Report - <Candidate Name> - <YYYY-MM-DD>.
- Body includes total jobs found, Naukri count, report generation time, and a short disclaimer that listings must be verified before applying.
- Attach the Excel workbook; optionally archive a copy in Google Drive.
- Retry transient email failures up to three times with backoff.
- If email fails permanently, mark run as failed_email and alert the operator.

10. Deployment Architecture

For 1-50 users, the recommended deployment is a single always-on backend process. Avoid Kubernetes, distributed queues, and managed databases until real usage proves they are needed.

Option	Monthly Cost	Fit
Render/Railway/Fly small worker	Free to low paid tier	Easy deployment; confirm always-on worker availability and sleep behavior.
Low-cost VPS	Low paid	Best control for Playwright browsers and scheduled background runs.
GCP Cloud Run scheduled job	Low if usage small	Good for stateless scheduled runs; browser scraping may need tuning.
GitHub Actions cron	Free/limited	Useful for prototypes, but weaker for long browser runs, secrets, and reliability.

Recommended MVP Deployment

- One low-cost VPS or always-on worker service.
- Python backend process with APScheduler.
- SQLite database stored on persistent disk with daily backup.
- Environment variables for Google service account, SMTP credentials, run cadence, and portal limits.
- Playwright browser dependencies installed during deployment.
- Health endpoint or heartbeat log checked daily by the operator.

11. Security, Privacy, and Compliance

Portal Compliance Requirement

Do not encode 'scraping is allowed' as a blanket technical assumption. Each portal agent must have a compliance note, rate-limit policy, and disable switch. The Naukri MVP should still use conservative scraping behavior and should be reviewed against current terms before launch.

- Use a Google service account with least-privilege access to only the intake Sheet and resume Drive folder.
- Store secrets in environment variables or hosting secret manager, never in code or Sheets.
- Keep resumes in Google Drive; store only file IDs, parse hashes, and short metadata in SQLite.

- Mask email and phone values in logs where possible.
- Add explicit consent text to the Google Form covering resume parsing, job search automation, and email reports.
- Provide a simple pause/unsubscribe mechanism through a Sheet field or operator process.
- Review portal terms before production scraping. LinkedIn's public User Agreement, effective November 3, 2025, includes restrictions against bots, scraping, and automated access, so LinkedIn should not be treated the same as Naukri without legal/product review.

12. Observability and Operations

Run Status Model

Status	Meaning
queued	Candidate selected and waiting for processing.
running	Profile building or portal agent work is active.
success	Report generated and email sent.
partial_success	At least one non-critical step failed but a report was sent.
failed_parse	Resume could not be parsed and form fallback was insufficient.
failed_scrape	Portal agent failed and no reportable jobs were available.
failed_email	Report generated but could not be delivered.

Required Logs

- run_id, candidate_id, schedule trigger, started_at, completed_at, status.
- Parser status, parse confidence, extracted skill count, warnings.
- Agent status, query variants, pages visited, jobs found, jobs retained, selector errors.
- Report path, workbook row counts, email delivery attempts.
- Exception class, message, and short remediation hint.

Operator Commands

- Run one candidate by candidate_id.
- Run all due candidates.
- Dry-run a candidate without sending email.
- Regenerate and resend the last report.
- Pause or unpause a candidate.

13. Non-Functional Requirements

Category	Requirement
Capacity	Support 1-50 active candidates, with 1-3 concurrent candidate runs.

Performance	Single candidate run should finish within 10 minutes for Naukri-only MVP.
Reliability	A portal failure must not corrupt run state; retry transient failures.
Maintainability	Each portal agent must be isolated behind a common interface.
Security	No secrets in source code; least-privilege Google access.
Privacy	Resume files remain in Google Drive; no permanent raw resume text storage by default.
Cost	No paid APIs in MVP unless explicitly approved.

14. Implementation Phases

Phase	Deliverable	Exit Criteria
0	Local prototype	Read sample Sheet row, parse sample PDF, create dummy Excel, send test email.
1	Google integration	Service account reads Sheet and Drive uploads reliably.
2	Profile builder	Candidate Profile Object generated with parser fallback and validation.
3	Naukri MVP agent	Scrapes and ranks listings for 3-5 test profiles without manual intervention.
4	Scheduler + reports	Morning scheduled run sends Excel reports and updates run state.
5	Hardening	Retries, logs, dedupe history, pause controls, and operator run commands.
6	Portal expansion	Add LinkedIn/Workday/Indeed only after compliance and selector strategy review.

15. Key Risks and Mitigations

Risk	Mitigation
Portal layout changes break scraping	Use isolated selectors per portal, fail gracefully, and log selector names.
Portal blocks automated access	Use conservative rates, disable switches, and review terms before each portal launch.
Resume is scanned/image-only	Detect low text extraction and fallback to form fields; request a text PDF if needed.
Email lands in spam	Use verified sender, plain professional body, and move to SES/SendGrid if needed.
Google API quota or auth error	Batch reads, cache per run, and alert operator when auth fails.
Candidate receives repeated jobs	Store job hashes and mark or suppress previously sent jobs.

Server sleeps during schedule	Use an always-on worker or VPS, not a sleeping free web service.
-------------------------------	--

16. Open Product and Technical Decisions

- Should the default cadence be Monday/Wednesday/Friday or every three days?
- What is the maximum number of jobs per report: 50, 100, or another value?
- Should previously sent jobs be hidden or shown with a Previously Sent flag?
- Should candidates be able to update/pause through a second Google Form?
- Should salary be a hard filter or a soft scoring preference?
- Should the system archive generated reports in Google Drive?
- What is the acceptable manual maintenance frequency for portal scraper fixes?
- Will any LLM be allowed later for description summarization or ranking, and what is the budget cap?

17. Acceptance Criteria

- A new Google Form submission appears in the Sheet and can be picked up by the backend.
- The backend can download the uploaded PDF resume and generate a Candidate Profile Object.
- A scheduled run at the configured morning time processes all due active candidates.
- NaukriAgent returns normalized job listings with apply links for a test candidate.
- The Excel report includes All Jobs, Naukri, and Run Summary sheets.
- The candidate receives the report by email with correct counts and attachment.
- The system records last_run_at, next_run_at, last_status, and last_error.
- If Naukri scraping fails, the run is marked failed_scrape or partial_success with a clear operator log.
- No raw secrets are committed to the repository.
- The system handles 50 candidates by processing them in bounded batches without launching excessive browser sessions.

18. Source Notes

Inputs used: Joblist v1 PRD draft dated 2026-06-18, owner process-flow clarification on 2026-06-19, and current public LinkedIn User Agreement review on 2026-06-19. Portal terms should be rechecked before production launch because they can change.